

LATIHAN SOAL

Soal No. 1

KAMUS UMUM

```
constant NMax: integer = 366
type TabSuhu: < Tab : array [1..NMax] of real;
               Neff : integer > { Bernilai  $\geq 0$ , merupakan
                                   nilai efektif array; Neff=0
                                   berarti array kosong }
```

Implementasi Fungsi SearchGtX

```
function SearchGtX (input T : TabSuhu, input X : real)  $\rightarrow$  boolean
{ Mencari apakah ada elemen dalam tabel T yang bernilai lebih besar
  dari X. Menghasilkan true jika ada, false jika tidak ada atau tabel
  kosong }
```

KAMUS LOKAL

```
i: integer { indeks untuk traversal tabel }
```

ALGORITMA

```
if (T.Neff = 0) then
   $\rightarrow$  false
else
  i  $\leftarrow$  1
  while (i  $\leq$  T.Neff) and (T.Tab[i]  $\leq$  X) do
    i  $\leftarrow$  i + 1
  { i > T.Neff or T.Tab[i] > X }
  if (i  $\leq$  T.Neff) then
     $\rightarrow$  true { ketemu elemen > X }
  else
     $\rightarrow$  false { tidak ketemu }
```

LATIHAN SOAL

Soal No. 2

KAMUS UMUM

```
constant NMax: integer = 100
type MatKul: < KodeMK : integer,
                NamaMK : string,
                Pengajar : string,
                JmlPeserta : integer[0..100] >
type TabMatKul: < TM : array [1..NMax] of MatKul,
                NELMT : integer > { nilai efektif [0..NMax], 0
                                     jika tabel kosong }
```

Implementasi Fungsi IdxPengajarMK

```
function IdxPengajarMK (input P : string, input T : TabMatKul) → integer
{ Mencari indeks terakhir di mana pengajar P ditemukan dalam tabel T.
  Menghasilkan indeks terakhir jika ketemu, 0 jika tidak ketemu atau
  tabel kosong }
```

KAMUS LOKAL

```
i: integer          { indeks untuk traversal tabel }
LastIdx: integer    { menyimpan indeks terakhir yang ditemukan }
```

ALGORITMA

```
if (T.NELMT = 0) then
    → 0
else
    LastIdx ← 0
    i ← 1
    while (i ≤ T.NELMT) do
        if (T.TM[i].Pengajar = P) then
            LastIdx ← i
        i ← i + 1
    → LastIdx
```

LATIHAN SOAL

Soal No. 3a

KAMUS UMUM

```
constant NMax: integer = 100
type TMahasiswa: < NIM : string,
                    Nama : string,
                    Nilai : integer >
type TabMhs: < TM : array [1..NMax] of TMahasiswa,
              Neff : integer >
```

Implementasi Fungsi UrutTabMhs

```
procedure UrutTabMhs (input/output TMhs : TabMhs)
```

```
{ Mengurutkan tabel mahasiswa berdasarkan nilai secara terurut
  mengecil menggunakan selection sort }
```

KAMUS LOKAL

```
i: integer      { indeks untuk traversal tabel }
pass: integer   { tahapan pengurutan }
temp: TMahasiswa { untuk pertukaran elemen }
IMax: integer   { indeks dengan nilai maksimum }
```

ALGORITMA

```
if (TMhs.Neff > 1) then
  pass traversal [1..TMhs.Neff-1]
  { Tentukan indeks dengan nilai maksimum di TMhs.TM[pass..TMhs.Neff] }
  IMax ← pass
  i traversal [pass+1..TMhs.Neff]
    if (TMhs.TM[IMax].Nilai < TMhs.TM[i].Nilai) then
      IMax ← i
  { Tukar TMhs.TM[IMax] dengan TMhs.TM[pass] }
  temp ← TMhs.TM[IMax]
  TMhs.TM[IMax] ← TMhs.TM[pass]
  TMhs.TM[pass] ← temp
  { TMhs.TM[1..pass] terurut mengecil berdasarkan Nilai }
{ Seluruh tabel terurut mengecil berdasarkan Nilai }
```

LATIHAN SOAL

Soal No. 3b

JAWABAN:

Ya, kasus diatas dapat diurutkan menggunakan **counting sort** dengan syarat dan pertimbangan.

SYARAT:

- 1) **Range nilai terbatas:** Atribut Nilai memiliki rentang [0..100], sehingga dapat dibuat array TabCount[0..100]
- 2) **Range nilai cukup kecil:** Nilai range yang besar akan menimbulkan keborosan dalam penggunaan memori
- 3) **Tipe data integer:** Nilai adalah integer, sesuai dengan persyaratan counting sort

KEKURANGAN & KELEBIHAN:

- **(Kelebihan):** Kompleksitas waktu $O(n + k)$ dimana $k = 101$ (range nilai) dan lebih efisien untuk data dengan range terbatas dan kecil.
- **(Kekurangan):** Membutuhkan memori tambahan untuk TabCount dan TabTemp, implementasi lebih kompleks karena harus mempertahankan data mahasiswa lengkap, dan tidak stabil secara natural (urutan mahasiswa dengan nilai sama mungkin berubah)

KESIMPULAN:

Counting sort dapat digunakan tetapi kurang praktis untuk kasus ini karena kita tidak hanya mengurutkan nilai integer, tetapi harus mempertahankan seluruh record mahasiswa. Selection sort lebih sederhana dan cocok untuk ukuran data yang tidak terlalu besar.

IMPLEMENTASI COUNTING SORT:

```
procedure UrutTabMhsCountingSort (input/output TMhs : TabMhs)
{ Mengurutkan tabel mahasiswa berdasarkan nilai secara terurut
  mengecil menggunakan counting sort }
```

KAMUS LOKAL

```
constant NMax: integer = 100
TabCount: array [0..100] of integer
TabTemp: array [1..NMax] of TMahasiswa
i, j, k: integer
```

ALGORITMA

```
if (TMhs.Neff > 1) then
  { Inisialisasi TabCount }
```

```

i traversal [0..100]
    TabCount[i] ← 0
{ Counting - hitung frekuensi setiap nilai }
i traversal [1..TMhs.Neff]
    TabCount[TMhs.TM[i].Nilai] ← TabCount[TMhs.TM[i].Nilai] + 1
{ Simpan data mahasiswa sementara }
i traversal [1..TMhs.Neff]
    TabTemp[i] ← TMhs.TM[i]
{ Pengisian kembali secara terurut mengecil (dari 100 ke 0) }
k ← 0
i traversal [100..0] { dari nilai tertinggi ke terendah }
    if (TabCount[i] ≠ 0) then
        { Cari mahasiswa dengan nilai i }
        j traversal [1..TMhs.Neff]
            if (TabTemp[j].Nilai = i) then
                k ← k + 1
                TMhs.TM[k] ← TabTemp[j]

```

LATIHAN SOAL

Soal No. 4

KAMUS UMUM

```
constant NMax: integer = 100
type TabInt: < Tab : array [1..NMax] of integer;
               Neff : integer > { indeks efektif tabel yang
                                   terdefinisi,  $0 \leq N \leq NMax$  }
```

Implementasi Fungsi InputTerurut

```
procedure InputTerurut (input/output T : TabInt, input X : integer)
```

```
{ Masukkan elemen X ke dalam tabel T dengan mempertahankan urutan
  dari kecil ke besar.
```

```
  PreCondition: T terurut membesar, T.Neff < NMax
```

```
  PostCondition: T tetap terurut membesar dengan elemen X
                  dimasukkan ke dalam TabInt T }
```

KAMUS LOKAL

```
  i: integer { indeks untuk mencari posisi dan pergeseran }
  pos: integer { posisi untuk menyisipkan X }
```

ALGORITMA

```
  if (T.Neff < NMax) then { masih ada tempat }
    if (T.Neff = 0) then
      { Tabel kosong, langsung masukkan }
      T.Tab[1] ← X
      T.Neff ← 1
    else
      { Cari posisi yang tepat untuk X }
      pos ← 1
      while (pos ≤ T.Neff) and (T.Tab[pos] < X) do
        pos ← pos + 1
      { pos adalah posisi untuk menyisipkan X }
      { Geser elemen-elemen mulai dari belakang }
      i ← T.Neff
      while (i ≥ pos) do
        T.Tab[i+1] ← T.Tab[i]
        i ← i - 1
      { Sisipkan X pada posisi yang tepat }
      T.Tab[pos] ← X
      T.Neff ← T.Neff + 1
```